

Advanced Machine Learning

NST-CS-AML502 | Semester 5 • 2026 | Complete Visual Cheatsheet & Revision Compendium

TOTAL LECTURES

12 Comprehensive Units

TOTAL PAGES

13 Pages (Cover + Notes)

COURSE TRACK

Semester 5 Examination Prep

TABLE OF CONTENTS & INDEX

UNIT	LECTURE TOPIC & SUMMARY	PAGE
L-01	Setting the Foundation: From Code to Learning	Page 02
L-02	The ML Project Lifecycle (Part 1)	Page 03
L-03	Simple Linear Regression Using OLS	Page 04
L-04	Multiple Linear Regression Using OLS	Page 05
L-05	Batch Gradient Descent for Multiple Linear Regression	Page 06
L-06	Stochastic & Mini-Batch Gradient Descent	Page 07
L-07	Regression & Classification Evaluation Metrics	Page 08
L-08	Polynomial Regression & Assumptions	Page 09
L-09	Bias, Variance & The Bias-Variance Tradeoff	Page 10
L-10	Feature Selection & Importance Scoring	Page 11
L-11	Dimensionality Reduction & PCA Mathematical Rigor	Page 12
L-12	Regularization: L1 (Lasso) & L2 (Ridge)	Page 13

1 Course Introduction & Syllabus

1.1 Meet the Faculty

- Shobhit Johri, AI/ML Faculty, NST - Rishihood Univ.
- B.Tech (IIT Roorkee) | Math Research Analyst @ Turing
- Specializes in Mathematics & ML Problem Solving.

1.2 Logistics & Grading Structure (12 Weeks)

Component	Weightage
Contest (Only 1)	10%
Lecture & Lab Assignments	5%
Projects & Viva	20%
Mid Semester Exam	25%
End Semester Exam	40%

Prerequisites

- Mathematics: Probability & Statistics, Linear Algebra, Calculus
- Python: Programming + Libraries (NumPy, Pandas, Matplotlib)
- Fundamentals: General AI & basic ML concepts.

2 Traditional Code vs. Machine Learning

2.1 The Rigidity of Explicit Rules

- Spam filter rules: block "FREE", "WINNER" → works initially.
- Spammers change spelling: "FR33", "W1NNER" → program breaks.

⇒ Traditional programming is structurally rigid.

Depends on human engineers writing explicit rules.

Environment shifts → Traditional program fails (no adaptation).

2.2 The Software Paradigm Flip



3 The AI, ML, and DL Ecosystem

3.1 Artificial Intelligence (AI)

- Broad field of CS → create systems that mimic human-like intelligence (reason, solve, perceive, decide autonomously).

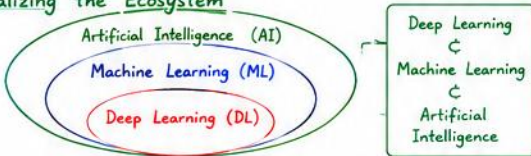
3.2 Machine Learning (ML)

- Subset of AI → develops math models/algorithms that learn patterns & insights from data (no hardcoded rules).

3.3 Deep Learning (DL)

- Subfield of ML → uses multi-layered Neural Networks.
- Automatically learns representations from raw, unstructured data (images, audio, text, etc.).
- Solves feature-engineering bottleneck of standard ML.

Visualizing the Ecosystem



4 Labelled vs. Unlabelled Data

4.1 Definition of Data Types

- Labelled Data:** Input features + explicit target/answer (ground truth). Example: 50,000 emails labelled "spam"/"not spam".
- Unlabelled Data:** Only input features, no target/labels. Example: 200,000 chest X-rays with no annotations.

4.2 Cost of Labeling

THE BOTTLENECK OF LABELLED DATA

- Requires human domain experts → expensive & time-consuming.
- Examples: Radiologists (X-rays), Lawyers (contracts), Linguists (audio transcription).
- Drives organizations to use unsupervised / semi-supervised methods.

Key Insight: More data ≠ better model if data is unlabelled and task needs supervision.

5 The Four Learning Strategies

5.1 Supervised Learning

- Uses labelled data (inputs + labels).
- Learns mapping function $f(x) \rightarrow y$ to predict labels for unseen data.
- Example: Predict housing prices from size, location.

5.2 Unsupervised Learning

- Uses unlabelled data only.
- Discovers hidden structure, patterns, groupings.
- Example: Market segmentation, topic discovery.

5.3 Semi-Supervised Learning

- Combines small labelled data + large unlabelled data.
- Uses labelled set to guide & propagate knowledge to unlabelled set.

Case Study: Google Photos



5.4 Reinforcement Learning

- No static dataset.
- Agent interacts with environment → takes actions.
- Gets feedback via Rewards (+) / Penalties (-).
- Goal: Learn policy π that maximizes cumulative reward.
- Example: Drone learns to fly → + for stable flight, - for crashes.

Input (Data)

↓

Process (Learn)

↓

Output (Decision/Action)

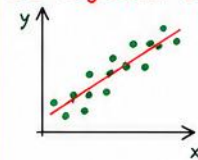
↓

Feedback (Improve)

↓

6 Core Machine Learning Tasks

6.1 Regression (Continuous Scalars)

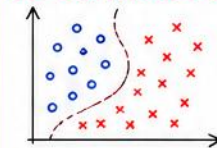


- Target output is continuous numerical value.
- Answers: "How much?", "How many?", "What value?"
- Example: Uber Surge Pricing → outputs continuous fare multiplier (1.4x, 2.1x, ...).

Common Algorithms:

- Linear Regression
- Support Vector Regression (SVR)
- Decision Tree Regression

6.2 Classification (Categories)

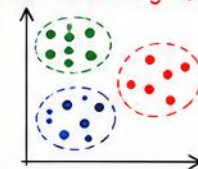


- Target output is discrete class label.
- Goal: Learn decision boundary between classes.
- Example: Medical risk screening → low-risk / high-risk.

Common Algorithms:

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Support Vector Machines (SVM)
- Naive Bayes

6.3 Clustering (Statistical Grouping)



- Unsupervised task → groups similar points.
- Points in cluster = similar; different clusters = dissimilar.
- Example: Netflix taste-clusters → personalized recommendations.

Common Algorithms:

- K-Means Clustering
- DBSCAN
- Gaussian Mixture Models (GMM)

1. DEFINING THE PROBLEM: THE ML BLUEPRINT

1.1 Translating Vague Human Problems

Phase 1: Translate messy human problem → explicit mathematical framework using a 5-step blueprint.

Step	What You Do	Example
1.	Understand the Problem Isolate the exact friction point.	Users canceling premium subscriptions (Churn).
2.	Set Quantifiable Goals Define a measurable target.	Reduce churn by 10% over two quarters.
3.	Assess ML Feasibility Ask: do we need ML? Do we have data? Can SQL solve it?	If a simple SQL query can solve the logic, skip ML.
4.	Identify Constraints Evaluate budget, latency limits, and privacy laws.	In-app predictions must run in < 10ms on a mobile chip.
5.	Stakeholder Alignment Bring PMs, devs, and executives to an agreement.	Share a common definition of "success."

— BLUEPRINT IN ACTION —

Vague: "We want to improve our online retail store."

Rigorous: "Using historical clickstreams and purchase data, predict which users will stop shopping within 3 months."

★ **CRITICAL TAKEAWAY:** Never write a line of ML code until you have a quantified target, confirmed data access, and stakeholder agreement on metrics.

1.2 Constraints & Alignment

Operational constraints can break an ML project before coding starts.

- e.g., GDPR rules or data collection limitations might make it illegal to use personal data for training models → project infeasible from the start.

CHECK BEFORE YOU CODE

- ☒ Quantified target defined
- ☒ Data access confirmed
- ☒ ML needed (SQL can't solve it)
- ☒ Constraints evaluated
- ☒ Stakeholders aligned on success metrics

2. DATA COLLECTION: SOURCING & STRUCTURES

2.1 Sourcing production data

- Internal Warehouses** (SQL databases): Customer transactions, purchase history, system logs.
- Public Repositories:** Open data from Kaggle, UCI ML Repository, etc.
- APIs:** Real-time streaming data (weather, stock prices, news feeds).
- Web Scraping:** Automated extraction from web (competitor pricing, reviews).
- Manual Labeling:** Human annotators labeling raw data (e.g., radiologists labeling tumors).

2.2 Data structures

Structured
Rigid table format (rows, columns)
e.g., SQL databases

Unstructured
No strict tabular layout (images, video, audio, text reviews)

Semi-structured
Tags/key-value without rigid schema (JSON, XML, logs)

Time-series
Data points at successive uniform time intervals (stock daily prices, hourly weather)

2.3 Categorical vs. Numerical Feature classification

Type	Description	Examples
Categorical Nominal	No mathematical rank or order	Blood type (A, B, AB, O), Eye color, City
Categorical Ordinal	Categories with rank or order	Education (HS < Bachelor < Master < PhD), Ratings
Numerical Discrete	Countable integer values	No. of rooms, items in a cart, clicks
Numerical Continuous	Infinitely divisible measurements	Income, temp., weight, height, time

3. EXPLORATORY DATA ANALYSIS (EDA)

3.1 Diagnostic Checklist & Analysis levels

EDA CHECKLIST

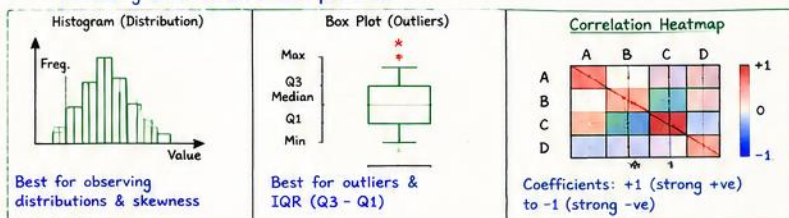
- ☒ **Dataset Structure:** columns, types, counts
- ☒ **Missing Values:** empty values? missing at random?
- ☒ **Outliers:** extreme anomalies (error vs. rare event)
- ☒ **Duplicates:** identical records from collection bugs
- ☒ **Distributions:** skewness (bell curve vs. skewed)
- ☒ **Relationships:** correlations, redundant features
- ☒ **Class Imbalance:** e.g., 99% non-fraud vs 1% fraud

LEVELS OF ANALYSIS

- Univariate:** one feature alone (distributions, skew, outliers) → Histograms, Box Plots
- Bivariate:** relationship between two features → Scatter Plots, Heatmaps
- Multivariate:** interactions across multiple features → Pair Plots

3.2 WHY CLASS IMBALANCE & Correlations

Visualizing data reveals hidden patterns.



4. DATA PREPROCESSING: CLEANING & SCALING

4.1 Handling Missing & Inconsistent Values

MISSING DATA STRATEGIES

- Deletion:** drop rows/cols. Use only if <1-2% missing or col >80% empty. Else → bias.
- Imputation:** fill with statistical values (mean/median for num., mode for cat.).

INCONSISTENT DATA

- Mixed Units:** kg & lbs → standardize to one.
- Types:** 'California' vs. 'Californi'a' → Regex or Levenshtein matching.
- Duplicates:** same record twice → remove.
- Inconsistent Labels:** Yes, Y, True, 1 → map all to single format (e.g., boolean).

4.2 Hand-worked scaling

(Min-Max, Z-score, Robust)

Scaler	Formula	Best Used When	Drawback / Risk
Min-Max (Normalization)	$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$	NN; bounded range [0, 1]; non-Gaussian.	One major outlier squashes all typical values near 0.
Z-Score (Standardization)	$Z = \frac{X - \mu}{\sigma}$	Normal dist.; mean → 0, std. dev. → 1.	Assumes approx. Gaussian dist.
Max Absolute	$X_{scaled} = \frac{X}{\max(X)}$	Sparse data; preserves zeros (output in [-1, 1]).	Sensitive to extreme outliers.
Robust Scaler	$X_{scaled} = \frac{X - \text{Median}}{IQR}$	Heavy outliers; uses Median & IQR.	Ignores outer, distribution data.

HAND COMPUTATION (ROBUST SCALING)

Given dataset X = [100, 150, 200, 250, 50000]

- Sorted list: [100, 150, 200, 250, 50000]
- Median (Q2) = 200
- Q1 (median of [100, 150]) = 125
- Q3 (median of [250, 50000]) = 25125
- IQR = Q3 - Q1 = 25125 - 125 = 25000
- Scale X = 1000: $\frac{100 - 200}{25000} = -0.004$
- Scale X = 50000: $\frac{50000 - 200}{25000} = 1.992$

Note: Outlier (50000) didn't break scaling limits. IQR protects typical data values.

5. CATEGORICAL ENCODING STRATEGIES

5.1 Label, Ordinal, One-Hot & Binary mappings

- Label Encoding:** map unique categories to unique integers. Best for target labels (Spam=1, Ham=0).
- Ordinal Encoding:** preserve rank order (Low=0, Med=1, High=2).
- One-Hot Encoding:** create N binary columns where a single 1 indicates the category. Best for unordered nominal data with low cardinality.
- Binary Encoding:** map to integers, then convert to binary digits → split into columns. Best for high-cardinality nominal data.

5.2 Curse of Dimensionality & Space comparisons

One-hot encoding high-cardinality features (e.g., 1000 cities) creates 1000 sparse columns → Curse of Dimensionality. Binary encoding resolves this space issue.

— SPACE COMPLEXITY: ONE-HOT vs. BINARY —

For 1,000 categories:

One-Hot Encoding
Creates 1,000 columns

VS.

Binary Encoding
Creates only $\lceil \log_2(1000) \rceil$
≈ 10 columns

EXAMPLE: BINARY ENCODING FOR [RED, GREEN, BLUE]

Map integers: Red=1 (01), Green=2 (10), Blue=3 (11)

Input	Bit 1	Bit 0
Red (1)	0	1
Green (2)	1	0
Blue (3)	1	1

Saves space while preserving information with fewer columns.

Simple Linear Regression Using the OLS Method

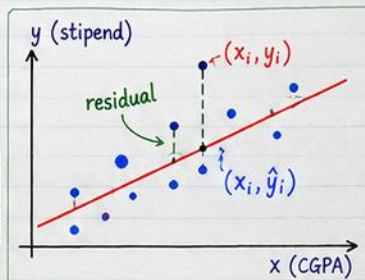
1 The Prediction Problem

- Given a dataset of student CGPAs and their internship stipends, we want to predict the stipend for a new student.
- When data follows a roughly linear pattern, we model it with a straight line.

$$\hat{y} = mx + c$$

$\hat{y}$ = predicted output (stipend)
 x = input feature (CGPA)
 m = slope (how steeply output changes)
 c = intercept (predicted output when $x = 0$)

- Since we learn from labeled examples (input + output pairs), this is **Supervised Learning**.

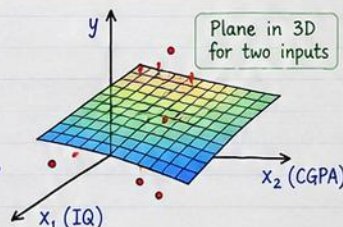


Extension to Multiple Inputs

- With two input features (e.g. IQ and CGPA), the model becomes a plane:

$$\hat{y} = m_1x_1 + m_2x_2 + c$$

- Same mathematical challenge applies: out of infinitely many possible planes, we need to pick the best one using an objective criterion. This lecture focuses on the single-input case.



2 Residuals & The Error Concept

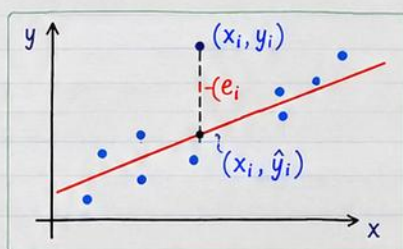
- For each data point (x_i, y_i) , the model predicts:

$$\hat{y}_i = mx_i + c$$

- The mismatch between actual and predicted value is called the **residual** (or error) at that point:

$$e_i = y_i - \hat{y}_i = y_i - (mx_i + c)$$

- Residuals appear as vertical gaps from each data point to the prediction line. We measure vertical distances because x is treated as fixed and we only predict y .



3 Three Loss Function Attempts

(a) Signed Error (Fails: Cancellation)

$$\text{Total Signed Error} = \sum e_i = \sum (y_i - \hat{y}_i)$$

- Positive and negative errors cancel each other.
- A total of 0 does not mean a perfect model.
- Hence, not a reliable loss function.

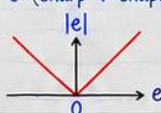
Example:

$e_1 = 10, e_2 = -10$
 Total = $10 - 10 = 0$
 (but errors are large)

(b) Absolute Error (Robust but Not Differentiable)

$$\text{Total Absolute Error} = \sum |e_i| = \sum |y_i - mx_i - c|$$

- Solves cancellation problem.
- Robust to outliers (penalized linearly).
- Not smoothly differentiable at $e = 0$ (sharp V-shape).

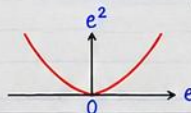


(c) Squared Error — The OLS Choice

$$E(m, c) = \sum e_i^2 = \sum (y_i - mx_i - c)^2$$

- Always non-negative.
- Differentiable everywhere (smooth parabola).
- Quadratic penalty for large errors.
- Trade-off**: sensitive to outliers.

Error (e)	$ e $	e^2
1	1	1
2	2	4
10	10	100



4 Closed-Form vs. Iterative Solutions

Closed-Form Solution

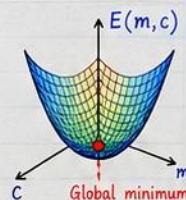
- Exact mathematical formula.
- Plug in data → get optimal parameters directly.
- OLS for simple linear regression is this type.

Iterative Solution

- Start with initial guesses.
- Update step-by-step until error is sufficiently small.
- Gradient Descent is this type (introduced in a future lecture).

The Convex Error Surface

- The squared error function $E(m, c)$ creates a 3D surface.
- For simple linear regression, this surface is a convex bowl.
- The bottom is the global minimum (best m, c).
- At the minimum, the surface is flat in both directions:



$$\frac{\partial E}{\partial c} = 0 \quad \text{and} \quad \frac{\partial E}{\partial m} = 0$$

5 OLS Derivation: Intercept c & Slope m

(a) Deriving Intercept c (Equation 1)

$$\frac{\partial E}{\partial c} = \sum 2(y_i - mx_i - c)(-1)$$

Set to zero and divide by -2:

$$\sum (y_i - mx_i - c) = 0$$

Separate summation: $\sum y_i - m \sum x_i - nc = 0$

Divide by n and use means $\bar{x}, \bar{y}$.

$$c = \bar{y} - m\bar{x}$$

(b) Deriving Slope m (Equation 2)

① Substitute $c = \bar{y} - m\bar{x}$ into $E(m, c)$.

② Take $\frac{\partial E}{\partial m}$ and set to zero.

③ Let $a_i = x_i - \bar{x}$ (deviation in x)
 $b_i = y_i - \bar{y}$ (deviation in y)

$$m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

Key Insight:

- Numerator: how x and y co-vary (covariance numerator).
- Denominator: how much x varies (variance).
- If all x_i are identical → denominator = 0 (slope undefined).

6 Final OLS Model & Outlier Sensitivity

Complete OLS Procedure

- Compute means $\bar{x}$ and $\bar{y}$.
- Compute slope: $m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$
- Compute intercept: $c = \bar{y} - m\bar{x}$
- Final model: $\hat{y} = mx + c$

Outlier Impact: Absolute vs Squared Loss

- Consider an extreme outlier: 10 study hours but only 20 marks (others ~80).
- Absolute Error**: penalty grows linearly → line shifts a little.
- Squared Error (OLS)**: penalty grows quadratically → line shifts significantly toward the outlier.
- Trade-off**: squared error is mathematically elegant but sensitive to outliers.

Key Insight:

- The OLS best-fit line always passes through the mean point $(\bar{x}, \bar{y})$.

Advanced Machine Learning

Lecture-04 : Multiple Linear Regression Using the OLS Method

① From SLR to MLR

1.1 The MLR Prediction Equation

- In SLR, we used a single feature (CGPA) to predict y .
- In real-world problems, we use multiple features (e.g., IQ, attendance, project scores, CGPA etc.).

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_m x_m$$

- m = number of input features
- n = number of data points (rows)
- β_0 = intercept (bias term)
- $\beta_1, \beta_2, \dots, \beta_m$ = learnable parameters (slopes)
- $\hat{y}$ = predicted output, y = actual output

★ Key Insight:

- This is the equation of a **hyperplane** in m -dimensional space.
- Adding more features does not change the structure, it just adds more terms.

1.2 Predictions for Every Data Point

- We have one equation template but n data points.
- For each row i :

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_m x_{im}$$

- Same parameters ($\beta_0, \beta_1, \dots, \beta_m$) for every row.
- Only feature values x_{ij} change from row to row.
- Stacking all predictions gives a prediction vector $\hat{y}$ of size $n \times 1$.

② Matrix Form: $\hat{y} = X\beta$

2.1 The Design Matrix X

- Write n equations in matrix form.
- X has n rows (data points) and $m+1$ columns (1 intercept column + m feature columns).

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1m} \\ 1 & x_{21} & x_{22} & \dots & x_{2m} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{nm} \end{bmatrix} \quad \begin{array}{l} \text{First column} \\ \text{all 1's to} \\ \text{add intercept} \\ \beta_0 \text{ to every} \\ \text{prediction.} \end{array} \quad n \times (m+1)$$

2.2 Dimensions & Compact Representation

Quantity	Notation	Dimension	Description
Design Matrix	X	$n \times (m+1)$	Features (with 1's)
Parameter vector	β	$(m+1) \times 1$	$[\beta_0, \beta_1, \dots, \beta_m]^T$
Prediction vector	$\hat{y}$	$n \times 1$	Predicted outputs
Actual output	y	$n \times 1$	True outputs

- The complete system:

$$\hat{y} = X\beta$$

★ Key Insight:

- Matrix notation compresses all n prediction equations into one line.
- X is the design matrix, β is the parameter vector, $\hat{y}$ is the predicted vector.

③ Residual Vector & Loss Function

3.1 Residuals as a Vector

- Residual (error) at each point i :

$$e_i = y_i - \hat{y}_i$$

- Stacking all residuals: $e = y - \hat{y} = y - X\beta$
- e is an $n \times 1$ vector.
- Each element e_i is the error at one data point.

3.2 Loss Function (Total Squared Error)

- Total squared error is the sum of squared residuals:

$$L(\beta) = \sum_{i=1}^n e_i^2 = e^T e = (y - X\beta)^T (y - X\beta)$$

★ Key Insight:

- The dot product $e^T e$ automatically computes the sum of squared residuals.
- $L(\beta)$ takes β as input and returns one scalar (total error).

④ Expanding the Loss Function

4.1 Transpose & Expand

- Using $(y - X\beta)^T = y^T - \beta^T X^T$

$$\begin{aligned} L(\beta) &= (y - X\beta)^T (y - X\beta) \\ &= (y^T - \beta^T X^T)(y - X\beta) \end{aligned}$$

4.2 Three-Term Expanded Form

$$L(\beta) = y^T y - 2y^T X\beta + \beta^T X^T X\beta$$

- $y^T y$: constant w.r.t. β
- $-2y^T X\beta$: linear in β
- $\beta^T X^T X\beta$: quadratic in β

⑤ Three Matrix Calculus Identities

To differentiate the expanded loss, we need three identities.

5.1 Identity 1 - Constant Term

- If $f(\beta) = c$ (scalar constant), $\frac{\partial(c)}{\partial\beta} = 0$

- Applied to our loss: $y^T y$ does not depend on β .

5.2 Identity 2 - Linear Form

- If $f(\beta) = a^T \beta$ (a constant vector a),

$$\frac{\partial(a^T \beta)}{\partial\beta} = a$$

- Applied to our loss: $-2y^T X\beta \rightarrow \frac{\partial(-2y^T X\beta)}{\partial\beta} = -2X^T y$

5.3 Identity 3 - Quadratic Form

- If $f(\beta) = \beta^T A \beta$ (A symmetric, $A = A^T$),

$$\frac{\partial(\beta^T A \beta)}{\partial\beta} = 2A\beta$$

- Applied to our loss: $\beta^T X^T X\beta$. Note $X^T X$ is symmetric.

$$\frac{\partial(\beta^T X^T X\beta)}{\partial\beta} = 2X^T X\beta$$

⑥ Normal Equation & Closed-Form Solution

6.1 Gradient = 0 $\rightarrow$ Normal Equation

- $\frac{\partial L}{\partial\beta} = 0 - 2X^T y + 2X^T X\beta = 0$

$$X^T X\beta = X^T y \quad (\text{Normal Equation})$$

6.2 The OLS Closed-Form Formula

- Multiply both sides by $(X^T X)^{-1}$:

$$\beta^* = (X^T X)^{-1} X^T y$$

★ Key Insight:

- This is the **closed-form OLS solution** for MLR.
- Directly computes optimal β from data.
- No iteration, no guessing.
- Works well for small m .

⑦ Limitations & SLR vs MLR Comparison

7.1 Multicollinearity & Invertibility

- $(X^T X)$ must be invertible $\Rightarrow \det(X^T X) \neq 0$.
- Requires columns of X to be **linearly independent**.
- If a feature is a **linear combination of others** $\rightarrow X^T X$ becomes **singular** (inverse doesn't exist).
- This is called **multicollinearity**.

7.2 Computational Cost $O(m^3)$

- $X^T X$ is $(m+1) \times (m+1)$.
- Matrix inversion via Gauss-Jordan: $O(k^3)$.
- For $m = 10,000$ features $\rightarrow 10^{12}$ operations (infeasible).
- For large m , use iterative methods (e.g., Gradient Descent).

Complete Derivation Path (8 Steps)

Step	Action	Result	SLR vs MLR Quick Reference		
1	Write MLR prediction	$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_m x_{im}$	Aspect	SLR (Lecture 3)	MLR (Lecture 4)
2	Stack as vector	$\hat{y} = X\beta$	Notation	m, c	$\beta_0, \beta_1, \dots, \beta_m$
3	Define residual	$e = y - X\beta$	Model	$\hat{y} = mx + c$	$\hat{y} = X\beta$
4	Total squared error	$L = (y - X\beta)^T (y - X\beta)$	Error	$\sum (y_i - mx_i - c)^2$	$(y - X\beta)^T (y - X\beta)$
5	Expand loss	$L = y^T y - 2y^T X\beta + \beta^T X^T X\beta$	Minimize	$\frac{\partial L}{\partial m} = 0, \frac{\partial L}{\partial c} = 0$	$\frac{\partial L}{\partial \beta} = 0$
6	Three identities	$\frac{\partial(c)}{\partial \beta} = 0, \frac{\partial(a^T \beta)}{\partial \beta} = a$ $\frac{\partial(\beta^T A \beta)}{\partial \beta} = 2A\beta$	Solution	m & c formulas	$\beta^* = (X^T X)^{-1} X^T y$
7	Differentiate, set $\nabla L = 0$	$X^T X\beta = X^T y$	Geometry	Line	Hyperplane
8	Solve for β^*	$\beta^* = (X^T X)^{-1} X^T y$	Condition	$\sum (x_i - \bar{x})^2 \neq 0$	$\det(X^T X) \neq 0$
			Limitation	1 feature only	$O(m^3)$ inversion cost

Batch Gradient Descent for Multiple Linear Regression

1 Why OLS is not enough

The closed-form OLS solution is exact:

$$\theta^* = (X^T X)^{-1} X^T y$$

However, there are two major limitations:

1 Computational cost:

- Directly inverting the matrix has computational cost $\approx O(m^3)$.
- For millions of features, the inverse becomes computationally impossible.

2 Invertibility and numerical stability:

- The formula works only when $X^T X$ is invertible (determinant $\neq 0$).
- This requires columns of X to be linearly independent.

Key Insight: If two features are exact duplicates, or feature is a linear combination of another, $X^T X$ becomes singular, so the inverse does not exist.

When $X^T X$ is singular?

- Two features are duplicates
- One feature is a linear combination of another
- Columns are linearly dependent

Example: $x_2 = 2x_1$

$$X = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \\ 4 & 8 \end{bmatrix}$$

Second column is $2 \times$ first column (linearly dependent)

$X^T X$ is singular (inverse does not exist)

2 Introduction to Gradient Descent

Objective: Reach the minimum of the loss function.

The parameter values at the bottom give the smallest prediction error.

Single-variable case:

- For a function $f(x)$, the derivative $f'(x)$ tells us the slope.
- To decrease the function, we move in the opposite direction of the derivative.

Multivariable case:

- The gradient ∇f is the vector of all partial derivatives.

Key Takeaway:

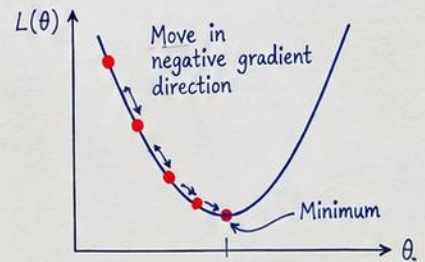
- The gradient points in the direction of steepest increase.
- The negative gradient points in the direction of steepest decrease.

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla L(\theta_{\text{old}})$$

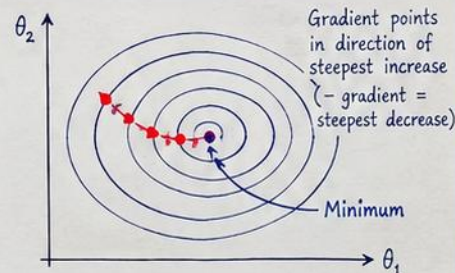
- θ : full parameter vector
- ∇L : vector of all partial derivatives
- α : learning rate

Warning: All partial derivatives must be computed using the old value of θ , and all parameters must be updated simultaneously.

Gradient Descent in 1D



Gradient Descent in 2D



3 Deriving the Batch Gradient Descent update for MSE

Mean Squared Error (MSE):

$$L(\theta) = \frac{1}{m} (X\theta - y)^T (X\theta - y)$$

where,

- X : design matrix ($m \times n$)
- θ : parameter vector ($n \times 1$)
- y : target vector ($m \times 1$)

Gradient:

$$\nabla L = \frac{2}{m} X^T (X\theta - y)$$

Batch Gradient Descent Update Rule:

$$\theta := \theta - \alpha \frac{2}{m} X^T (X\theta - y)$$

Key Insight: Why is this called **Batch Gradient Descent**? Because the gradient uses the full matrix X and the full target vector y . Every training example contributes to every single update step.

Lecture 06 - Stochastic & Mini-Batch Gradient Descent

Advanced
Machine
Learning

① Recap: Batch Gradient Descent (BGD)

- Objective: Minimize cost function $J(\theta)$ (e.g. MSE for linear regression)
- Compute gradient using all m training examples.
- Update parameters once per epoch (or once per full pass).

$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t) \quad \nabla J(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla J_i(\theta)$$

Characteristics:

- Stable direction (averages out noise)
- Good for small datasets
- Only 1 update per epoch
- High computational cost ($O(md)$)
- Needs full dataset in memory

② Stochastic Gradient Descent (SGD)

- Instead of using all examples, use **one randomly chosen** example i at a time.
- Update parameters immediately after computing gradient for that single example.
- Do this for every example in the dataset (one epoch = m updates).

$$\theta_{t+1} = \theta_t - \alpha \nabla J_i(\theta_t)$$

Here, $\nabla J_i(\theta_t)$ is the gradient computed using only the i -th example.

Key Points:

- 1 update per example (m updates per epoch)
- Very low cost per update ($O(d)$)
- Noisy gradient (high variance)
- Often converges faster in early stages
- Works well for large datasets and online learning

③ Mini-Batch Gradient Descent

- A compromise between BGD and SGD.
- Split the dataset into small groups called mini-batches of size b ($1 < b < m$).
- Compute the average gradient over each mini-batch and update parameters immediately.

$$g_k = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla J_i(\theta_t) \quad \theta_{t+1} = \theta_t - \alpha g_k$$

Why Mini-Batch?

- Less noisy than SGD
- More frequent updates than BGD
- Better use of vectorized computations (GPU/CPU)
- Works well in practice (most common)

④ Algorithm (Mini-Batch GD)

- Shuffle the dataset at the start of each epoch.
- Split into mini-batches of size b .
- For each mini-batch B_k :
 - Compute average gradient over the batch.
 - Update parameters immediately.
- Continue until every sample has been used once (one epoch).

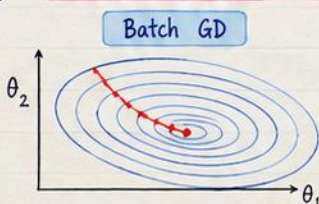
Pseudo Code:

```
for epoch = 1 ... E:
    shuffle(dataset)
    split into mini-batches  $B_1, \dots, B_K$ 
    for each batch  $B_k$ :
         $g = 0$ 
        for  $i$  in  $B_k$ :
             $g = g + \nabla J_i(\theta)$ 
         $g = g / |B_k|$ 
         $\theta = \theta - \alpha \cdot g$ 
```

⑤ Comparison

Property	Batch GD	Stochastic GD (SGD)	Mini-Batch GD
Gradient used	All m examples	1 example	b examples
Updates per epoch	1	m	m/b
Cost per update	$O(md)$	$O(d)$	$O(bd)$
Path to minimum	Smooth	Noisy (high variance)	Moderately noisy
Memory requirement	High (full data)	Low	Medium
Best for	Small datasets	Large datasets, online	Most common in practice

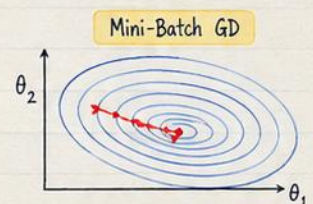
⑥ Gradient Descent Paths (Intuition)



Smooth and steady path towards minimum.



Noisy path but often reaches minimum faster in early stages.



Less noisy than SGD, faster than BGD.

Advanced Machine Learning

Lecture 07 - Regression and Classification Evaluation Metrics

1 Training Has Ended; Evaluation Begins

- The same expression can play both roles.
- MSE is a loss when it guides learning and an evaluation metric when it judges a finished model.

Training Loss	Evaluation Metric
- Used during training - Guides parameter updates - Minimized using optimization (e.g., Gradient Descent)	- Used after training - Judges a fixed model - Computed on validation or test data

2 Regression Error Metrics

- In regression, we measure how close the predicted value $\hat{y}$ is to the true value y .

Residual (error): $e_i = y_i - \hat{y}_i$

- Signed residuals show direction, but positive and negative values can cancel.

Mean Absolute Error (MAE)

- Average of absolute errors
- Easy to interpret
- Hides direction of error

$$MAE = \frac{1}{n} \sum_{i=1}^n |e_i|$$

Mean Squared Error (MSE)

- Average of squared errors
- Strongly penalizes large errors
- Useful for optimization

$$MSE = \frac{1}{n} \sum_{i=1}^n e_i^2$$

Root Mean Squared Error (RMSE)

- Square root of MSE
- Keeps large-error penalty
- Returns to the original unit of target variable

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n e_i^2}$$

Mean Absolute Percentage Error (MAPE)

- Scale-free (percentage error)
- Easy to interpret
- Undefined when $y = 0$

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{e_i}{y_i} \right|$$

3 R^2 and Adjusted R^2

- R-squared compares the leftover variation with the original variation.

$$R^2 = 1 - \frac{RSS}{TSS}$$

- RSS (Residual Sum of Squares) $= \sum_{i=1}^n (y_i - \hat{y}_i)^2$
- TSS (Total Sum of Squares) $= \sum_{i=1}^n (y_i - \bar{y})^2$

- R^2 tells us the proportion of variance in y explained by the model.
- Range: $(-\infty, 1]$, in practice: $0 \leq R^2 \leq 1$ (for reasonable models).
- A higher R^2 indicates a better fit.
- Adding a predictor cannot increase RSS, therefore training R^2 cannot decrease. Adjusted R^2 penalizes unnecessary features.

Adjusted R^2 :

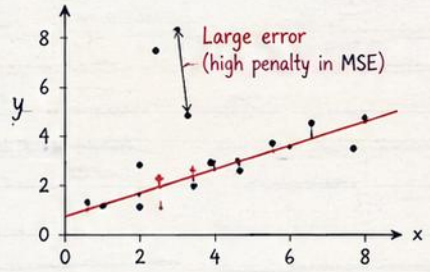
$$R^2_{adj} = 1 - (1 - R^2) \cdot \frac{n-1}{n-p-1}$$

n = number of samples
 p = number of predictors (features)

Key Points:

- Adjusted R^2 can decrease if a new feature does not improve the model.
- Useful for comparing models with different number of features.

Example: Effect of Different Metrics



Why different metrics?

- MAE: treats all errors equally.
- MSE/RMSE: penalizes large errors more (useful when large errors are costly).
- MAPE: useful when comparing across different scales, but not defined at $y = 0$.

Comparison of Metrics

Metric	Formula	Key Property
MAE	$\frac{1}{n} \sum_{i=1}^n e_i $	Easy to interpret, robust to outliers
MSE	$\frac{1}{n} \sum_{i=1}^n e_i^2$	Strongly penalizes large errors
RMSE	$\sqrt{\frac{1}{n} \sum_{i=1}^n e_i^2}$	Same unit as target variable
MAPE	$\frac{100}{n} \sum_{i=1}^n \left \frac{e_i}{y_i} \right $	Scale-free, undefined at $y = 0$

4 Classification Evaluation Metrics

- Accuracy tells how often the complete label was correct, but it does not identify the type or direction of error.

Confusion Matrix (Binary Classification)

Actual	Predicted	
	Positive	Negative
Positive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)

Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- Overall correctness of the model.

Precision

$$Precision = \frac{TP}{TP + FP}$$

- Among predicted positives, how many are correct?

Recall (True Positive Rate)

$$Recall = \frac{TP}{TP + FN}$$

- Among actual positives, how many are found?

F1 Score

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

- Harmonic balance of precision and recall.
- Useful when classes are imbalanced.

Lecture 12 - Polynomial Regression & Assumptions of Linear Regression

1 Polynomial Regression as Feature Transformation

- Standard linear regression: $y = \beta_0 + \beta_1 x$
- This fits a straight line (constant slope) and cannot capture curved relationships $\rightarrow$ leads to underfitting.

Idea: Feature Transformation

- Create new features using powers of x :

$$\phi(x) = (x, x^2)$$

- Model becomes: $y = \beta_0 + \beta_1 x + \beta_2 x^2$

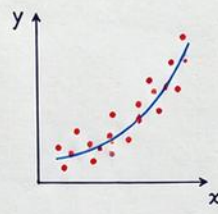
$\rightarrow$ Non-linear in x but linear in parameters $\beta_0, \beta_1, \beta_2$.
So we can still use OLS / Gradient Descent.

Example: (Internship Stipend)

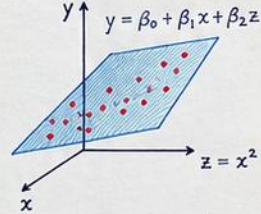
Student	Raw x	Transformed x^2	Target y (Stipend $\times 1000$)
A	2	4	24
B	4	16	42
C	6	36	62
D	8	64	91
E	10	100	128

3D Mental Model

- Let $z = x^2$. Then $y = \beta_0 + \beta_1 x + \beta_2 z$ represents a flat plane in 3D space (x, z, y) .
- When projected back onto the 2D curve $z = x^2$, it looks like a smooth parabolic curve in (x, y) space.



2D view: Parabolic curve
 $y = \beta_0 + \beta_1 x + \beta_2 x^2$



3D view: Linear plane
in (x, z, y) space

2 Polynomial Degree & Overfitting Risk

- Degree- n model:

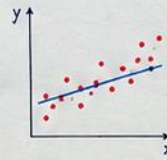
$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_n x^n = \beta_0 + \sum_{j=1}^n \beta_j x^j$$

- Degree n gives up to $n-1$ bends (inflection points).

Degree (n)	Model	No. of bends	Shape
1	$y = \beta_0 + \beta_1 x$	0	Straight line
2	$y = \beta_0 + \beta_1 x + \beta_2 x^2$	1	Parabola (U-shape)
3	$y = \beta_0 + \dots + \beta_3 x^3$	2	S-curve
n	$y = \beta_0 + \dots + \beta_n x^n$	$n-1$	Complex curve

Effect of Degree on Fit

Degree 1
(Underfitting)



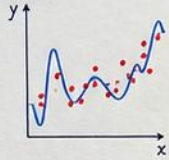
- High training error
- High validation error
- Too simple

Degree 2
(Good Fit)



- Low training error
- Low validation error
- Captures true trend

Degree 8
(Overfitting)



- Near-zero training error
- High validation error
- Fits noise

Key Points:

- Higher degree $\rightarrow$ lower training error (more parameters).
- But lower training error does NOT mean better model.
- Use cross-validation to select optimal degree n^* .
- Very high degree fits random noise $\rightarrow$ overfitting.

3 The 5 Assumptions of Linear Regression

Residual: $e_i = y_i - \hat{y}_i$

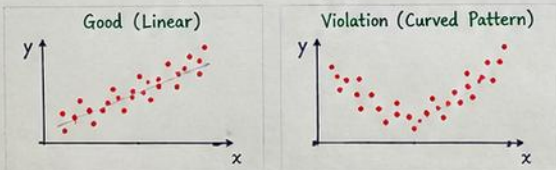
- These assumptions ensure that OLS estimates $\hat{\beta}$ are unbiased, minimum variance (BLUE) and valid for statistical inference.
- Important for inference (p-values, confidence intervals) and interpretability. Prediction is less affected (except multicollinearity).

3.1 Assumption 1: Linearity of Relationships

- Expected value of y is a linear combination of features.

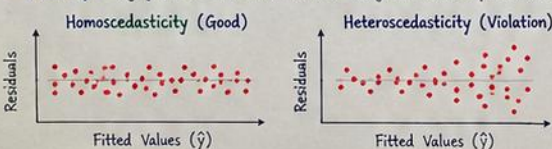
$$E[y|X] = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$$

- Check:** Scatter plots of y vs x and residuals vs fitted ($\hat{y}$).
- Residuals should be randomly scattered around 0.
- If violated:** Use non-linear transformations (log, sqrt) or add polynomial terms (x^2).



3.3 Assumption 3: Homoscedasticity (Constant Variance)

- $\text{Var}(e_i) = \sigma^2$ is constant across all predicted values $\hat{y}$.
- If variance changes $\rightarrow$ **Heteroscedasticity**.
- Consequences:** $\hat{\beta}$ remains unbiased but standard errors are incorrect $\rightarrow$ false statistical significance.
- Remedy:** $\log(y)$, Box-Cox transform or Weighted Least Squares (WLS).

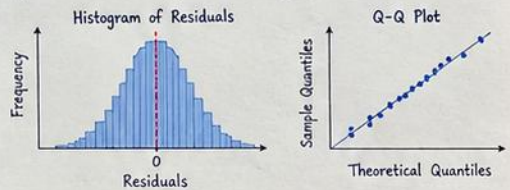


3.2 Assumption 2: Normality of Residuals

- Error terms e_i follow a normal distribution with mean 0 and constant variance σ^2 .

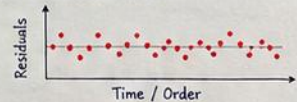
$$E[e_i] = 0, \quad e_i \sim N(0, \sigma^2)$$

- Check:** Histogram of residuals (bell curve) or Q-Q plot.
- Important for valid hypothesis testing (p-values, confidence intervals).



3.4 Assumption 4: No Autocorrelation of Errors

- Error terms are independent: $\text{Cov}(e_i, e_j) = 0$ for $i \neq j$.
- Common in time-series data.
- Check:** Plot residuals vs time/order. Durbin-Watson statistic $d \approx 2$ indicates no autocorrelation.



3.5 Assumption 5: No Multicollinearity

- Input features should be **independent** (not highly correlated).
- If X_1, X_2 are highly correlated $\rightarrow$ coefficients become unstable with large standard errors.
- Less impact on pure prediction, but harms interpretability/inference.
- Check:** Variance Inflation Factor (VIF).

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

R_j^2 : R^2 from regressing feature X_j on all other features.
 $\text{VIF} > 10 \rightarrow$ severe multicollinearity.

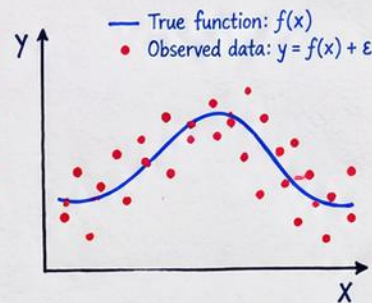
Lecture 08 : Bias, Variance and the Bias-Variance Tradeoff

1 Understanding Bias and Variance

$$Y = f(X) + \epsilon$$

- We do not know the true function $f(x)$.
- Using training data, we build a model $\hat{f}(x)$ which is an estimator of $f(x)$.
- If the training sample changes, the estimator $\hat{f}(x)$ also changes.
- Hence, $\hat{f}(x)$ is a random function.

- $f(x)$: true hidden relationship between input X and output Y . (deterministic function)
- ϵ : random noise with $E[\epsilon] = 0$ and variance σ^2 (irreducible error)

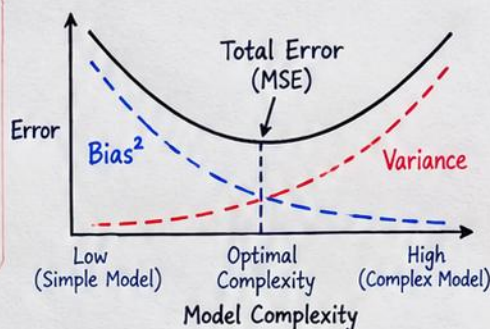


2 The Bias-Variance Tradeoff

- When we try to decrease the bias of a model, its variance automatically starts increasing.
- We aim to find the right model complexity where the total error is minimum.

Mean Squared Error (MSE) Decomposition

$$\text{MSE} = \underbrace{\text{Bias}^2}_{\substack{\text{Error due to} \\ \text{wrong assumptions} \\ \text{(simplistic model)}}} + \underbrace{\text{Variance}}_{\substack{\text{Error due to} \\ \text{sensitivity to} \\ \text{training data}}} + \underbrace{\text{Var}(\epsilon)}_{\substack{\text{Irreducible error} \\ \text{due to random} \\ \text{noise in data}}}$$



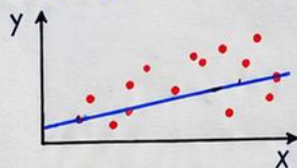
3 Diagnosing High Bias and High Variance

In practice, we look at training and validation accuracy (or error) to diagnose the problem.

High Bias (Underfitting)

Metric	Value (Example)
Training Accuracy	70% (Low)
Validation Accuracy	68% (Low, similar)

- Models are too simple.
- Cannot capture the underlying pattern.
- Leads to high error on both training and validation data.
- Example: Linear model for complex non-linear data.

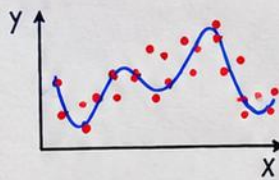


Underfitting (High Bias)

High Variance (Overfitting)

Metric	Value (Example)
Training Accuracy	99% (Very High)
Validation Accuracy	82% (Lower)

- Models are too complex.
- Fit the training data very well.
- Very sensitive to training data.
- Perform poorly on new, unseen data.
- Example: High degree polynomial.

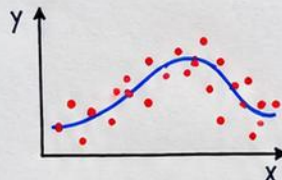


Overfitting (High Variance)

Good Fit (Balanced)

Metric	Value (Example)
Training Accuracy	88%
Validation Accuracy	85% (Close)

- Reasonably high accuracy on both.
- Small gap between training and validation accuracy.



Good Fit (Balanced)

4 Learning Curves Give Another Clue

A learning curve shows how the training and validation errors change as we increase the amount of training data.

High Bias (Underfitting)



- Both training and validation errors remain high, even after adding more data.
- Model is too simple.

High Variance (Overfitting)



- Training error is very low.
- Validation error is much higher but decreases as more data is added.
- Gap between the two errors.

Good Fit (Balanced)



- Both errors decrease and converge to a low value.
- Small gap between training and validation errors.

Lecture 09 : Feature Selection

1 Why Feature Selection?

- Process of choosing a smaller subset of **relevant** features while **preserving**, or ideally **improving**, performance on unseen data.
- Helps in removing **redundant** and **irrelevant** inputs without discarding genuinely useful signal.

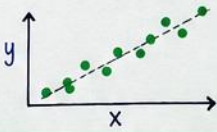
Key Idea :

Choose a smaller subset of relevant features while preserving (or improving) performance on unseen data.

Types of Features :

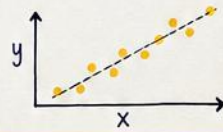
○ Relevant :

- Carries stable info. that helps predict the target.



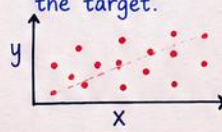
○ Redundant :

- Repeats information already carried by another feature.



○ Irrelevant :

- Varies but does not carry a stable relationship with the target.



Takeaway :

Remove redundant and irrelevant inputs without discarding genuinely useful signal.

2 Filter Methods

- Acts like an **admission gate**.
- Evaluates a feature using a simple structural or statistical rule.
- Fast and scalable** but usually **ignore interactions** among multiple features.

1 Duplicate Removal

- Removes exact copies of a column.

X ₁	X ₂	X ₃
1	1	4
2	2	5
3	3	6

 →

X ₁	X ₃
1	4
2	5
3	6

- If X₂ is duplicate of X₁ → remove it.

2 Variance Threshold

- Removes features that barely change.

$$\text{Var}(f_j) < \tau$$

Feature	Variance	
X ₁	0.0001	✗
X ₂	1.25	✓
X ₃	0.002	✗

Remove features with very low variance.

3 Pearson Correlation

- Removes features with low correlation to the target.

$$|r_{jY}| < 0.30$$

Feature	r (with Y)	
X ₁	0.76	✓
X ₂	0.12	✗
X ₃	-0.05	✗

Remove features with $|r_{jY}| < 0.30$.

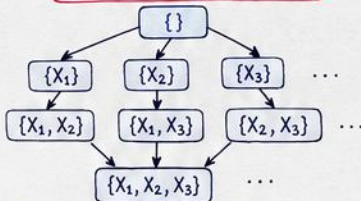
3 Wrapper Methods

- "Wraps" model training and validation around the feature-selection process.
- Directly optimizes performance for the chosen model.
- Computationally expensive and can overfit.**

1 Exhaustive Search

- Evaluates every nonempty subset of features.

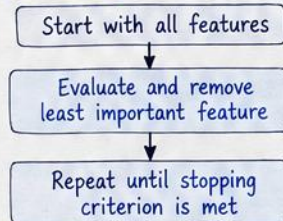
$$\text{Total subsets} = 2^P - 1$$



- Tries all possible combinations.
- Very expensive (feasible only for small p).

2 Backward Elimination

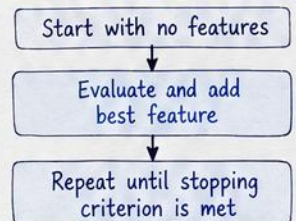
- Starts with all features, removes the worst one iteratively.



- Remove one feature at a time.
- Stop based on validation performance (e.g., no improvement).

3 Sequential Forward Selection

- Starts with no features, adds the best one iteratively.



- Add one feature at a time.
- Stop based on validation performance (e.g., no improvement).

4 Validation Discipline : Avoid the Leakage Trap

- Feature selection is part of teaching your model.
- It is not a harmless step you can do before properly splitting your data.



WARNING : Selecting features using the entire dataset and then splitting it leads to data leakage.

Right way :

Inside every cross-validation fold, perform feature selection using only the training portion of that fold.

Advanced Machine Learning - Lecture 10

Dimensionality Reduction & Principal Component Analysis (PCA)

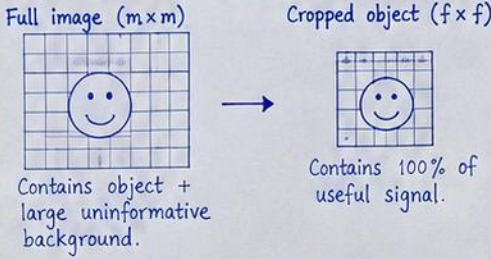
1. Why Dimensionality Reduction?

- When dataset has many features (p), each observation is a point in p -dimensional space.
- More features give richer inputs, but performance does not keep growing indefinitely.
- Beyond a certain point, unnecessary/redundant features add noise, inflate variance and increase computational cost ($O(p^2)$).

Curse of High Dimensionality

More dimensions $\Rightarrow$ data becomes sparse, distance measures become less meaningful and models may overfit.

Image Representation Example:



$p = m^2$ (all pixels) $\Rightarrow$ huge computation

Model accuracy plateaus at f^2 pixels.

So reducing dimensions can give same / better performance with less computation.

2. Feature Selection vs Feature Extraction

Method	Mechanism	Pros & Cons
Feature Selection	Keeps a subset of original features $\{x_1, x_2, \dots\}$ and discards others.	+ Preserves interpretability. - May lose information if signal is spread across features.
Feature Extraction (PCA)	Constructs new rotated orthogonal axes $PC_1, PC_2, \dots$ (linear combinations of all features)	+ Captures maximum variance (even diagonal spreads). - Components lose direct interpretability.

3. What is PCA?

Principal Component Analysis (PCA) is an unsupervised linear feature-extraction technique that finds a new set of orthogonal coordinate axes (principal components) along which the variance of the data is maximized.

4. Notation & Data Matrix Setup

n : number of training observations (rows)
 p : number of original features (columns)
 k : number of retained principal components ($k \leq p$)
 $X \in \mathbb{R}^{n \times p}$: original (uncentered) data matrix
 $\mu \in \mathbb{R}^{p \times 1}$: column mean vector $\mu_j = \frac{1}{n} \sum_{i=1}^n x_{ij}$
 $X_c \in \mathbb{R}^{n \times p}$: mean-centered data matrix
 $S \in \mathbb{R}^{p \times p}$: sample covariance matrix

5. Step 1: Mean Centering

$$X_c = X - 1\mu^T$$

- $1 \in \mathbb{R}^{n \times 1}$ is a column vector of ones.
- $1\mu^T$ is an $n \times p$ matrix replicating the mean row across all observations.

Why Mean Centering is Compulsory?

- PCA measures variance relative to the origin $(0,0,\dots,0)$.
- If not centered, the offset from the origin distorts the variance calculation and first PC points toward the mean instead of true spread.

6. Numerical Example (3 Houses, 2 Features)

Let X (Bedroom area f_1 , Washroom area f_2):

$$X = \begin{bmatrix} 2 & 1 \\ 4 & 3 \\ 6 & 5 \end{bmatrix}$$

Column means:

$$\mu = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

$$1\mu^T = \begin{bmatrix} 4 & 3 \\ 4 & 3 \\ 4 & 3 \end{bmatrix}$$

$$X_c = X - 1\mu^T = \begin{bmatrix} 2-4 & 1-3 \\ 4-4 & 3-3 \\ 6-4 & 5-3 \end{bmatrix} = \begin{bmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \end{bmatrix}$$

7. Standardization (When Scale Matters)

$$z_{ij} = \frac{x_{ij} - \mu_j}{s_j}$$

$$s_j = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \mu_j)^2}$$

- Use only centering: when features have similar units and original variances are meaningful.
- Use standardization: when features have different units or very different variance scales.

8. 1D Projection onto Unit Direction u

- Let $u \in \mathbb{R}^{p \times 1}$ be a unit vector ($\|u\|=1$).
- The projected scalar coordinate of centered observation x_i :

$$z_i = u^T x_i$$

9. Variance vs Reconstruction Error

By Pythagorean decomposition:

$$\|x_i\|^2 = z_i^2 + \|e_i\|^2$$

Since total norm $\sum \|x_i\|^2$ is fixed,

$$\max_{\|u\|=1} \text{Var}(z) \Leftrightarrow \min_{\|u\|=1} \sum_{i=1}^n \|e_i\|^2$$

10. Derivation of Variance Formula

$$\begin{aligned} \text{Var}(z) &= \frac{1}{n-1} \sum_{i=1}^n (u^T x_i)^2 \\ &= \frac{1}{n-1} \sum_{i=1}^n (u^T x_i)(x_i^T u) \\ &= u^T \left(\frac{1}{n-1} \sum_{i=1}^n x_i x_i^T \right) u \end{aligned}$$

$$\text{Var}(z) = u^T S u$$

$$S = \frac{1}{n-1} X_c^T X_c$$

(sample covariance matrix)

11. Sample Covariance Matrix

$$\text{Cov}(f_j, f_k) = \frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \mu_j)(x_{ik} - \mu_k)$$

$$S = \frac{1}{n-1} X_c^T X_c \quad (p \times p \text{ matrix of all pairwise covariances})$$

Running example:

$$S = \frac{1}{3-1} \begin{bmatrix} -2 & 0 & 2 \\ -2 & 0 & 2 \end{bmatrix} \begin{bmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 8 & 8 \\ 8 & 8 \end{bmatrix} = \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix}$$

12. Eigenvalues and Eigenvectors

Maximizing $u^T S u$ subject to $\|u\|=1$ gives:

$$Sv = \lambda v \Leftrightarrow |S - \lambda I| = 0$$

For running example:

$$\begin{vmatrix} 4-\lambda & 4 \\ 4 & 4-\lambda \end{vmatrix} = (4-\lambda)^2 - 16 = 0 \Rightarrow \lambda^2 - 8\lambda = 0$$

$$\lambda_1 = 8, \lambda_2 = 0$$

Eigenvectors:

$$\text{For } \lambda_1 = 8: v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad (\text{Overall House Size})$$

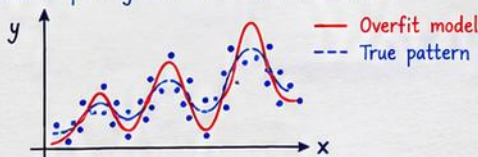
$$\text{For } \lambda_2 = 0: v_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} \quad (\text{Allocation Difference})$$

Lecture 11 : Regularization

L1 (Lasso) & L2 (Ridge) Regression

1. Motivation : Overfitting

- Unconstrained models often fit every noisy data point on training set.
- Training loss $\downarrow$ to zero but model memorizes noise instead of true pattern (Overfitting).
- Leads to poor generalization on test data.



2. Coefficient Magnitude as Overfitting Symptom

- Overfitted models have abnormally large coefficients ($w_j \rightarrow \infty$).
- A steep slope means tiny change in input Δx produces large change in prediction $\Delta \hat{y}$ (high variance).
- Creates hypersensitivity to small variations.

Example :

Model A (Gentle, Robust)

$$\hat{y} = 2x + 10$$

$$\Delta x = 0.5 \Rightarrow \Delta \hat{y} = 1.0$$

Model B (Overfit, High Variance)

$$\hat{y} = 30x - 18$$

$$\Delta x = 0.5 \Rightarrow \Delta \hat{y} = 15.0$$

3. What is Regularization ?

- Modification of loss function by adding a penalty term on model weights.
- Penalizes large weights and trades small increase in bias for large reduction in variance.

$$L_{\text{regularized}}(w) = L_{\text{data}}(w) + \lambda P(w)$$

Original loss

Regularization parameter ($\lambda \geq 0$)

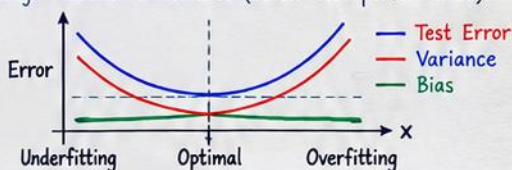
Penalty term on weights

Common choices for $P(w)$:

- L2 (Ridge) : $P(w) = \sum w_j^2$
- L1 (Lasso) : $P(w) = \sum |w_j|$

4. Bias - Variance Tradeoff

- Regularization helps control model complexity.
- Small increase in bias (slightly higher training error).
- Large reduction in variance (better test performance).



5. L2 Regularization : Ridge Regression

1. Ridge Loss Function (Simple Linear Regression)

- For $y = mx + c$, Ridge adds penalty on squared slope:

$$L_{\text{Ridge}}(m, c) = \sum_{i=1}^n (y_i - mx_i - c)^2 + \lambda m^2$$

2. 2D Closed-Form Slope Derivation

$$\text{Step 1 : } \frac{\partial L_e}{\partial c} = -2 \sum_{i=1}^n (y_i - mx_i - c) = 0$$

$$c = \bar{y} - m\bar{x}$$

Step 2 : Substitute $c = \bar{y} - m\bar{x}$ back into Loss

$$L_{\text{Ridge}}(m) = \sum_{i=1}^n [(y_i - \bar{y}) - m(x_i - \bar{x})]^2 + \lambda m^2$$

Step 3 : Differentiate w.r.t. m and set to 0

$$-2 \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) + 2m \sum_{i=1}^n (x_i - \bar{x})^2 + 2\lambda m = 0$$

Step 4 : Solve for m_{Ridge}

$$m_{\text{Ridge}} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2 + \lambda}$$

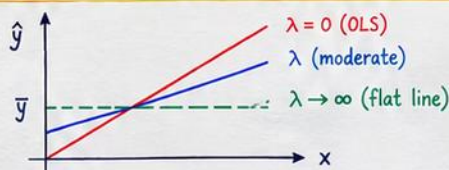
9. OLS vs Ridge vs Lasso Comparison

Feature	OLS Regression	Ridge (L2)	Lasso (L1)
Penalty Term	None ($\lambda = 0$)	$\lambda \sum w_j^2$	$\lambda \sum w_j $
Coefficients = 0 ?	No	No (shrinks near 0)	Yes (drives $w_j = 0$)
Feature Selection	No	No (retains all features)	Yes (automatic sparse selection)
Closed-Form Solution	$(X^T X)^{-1} X^T y$	$(X^T X + \lambda I)^{-1} X^T y$	No closed-form (Coordinate Descent)
Multicollinearity	Fails (Singular matrix)	Excellent (always invertible)	Selects one feature arbitrarily

6. Effect of λ on Ridge Slope

- $\lambda = 0$: Standard OLS slope (high variance if noisy).
- $\lambda > 0$: Denominator increases, shrinking slope towards zero.
- $\lambda \rightarrow \infty$: $m_{\text{Ridge}} \rightarrow 0$, producing a flat horizontal line $\hat{y} = \bar{y}$ (Underfitting).

Larger $\lambda \Rightarrow$ smaller slope $\Rightarrow$ more bias, less variance.



7. Multi-Feature Ridge & Key Properties

1. Matrix Form of Ridge Loss (p features)

$$L_{\text{Ridge}}(\beta) = (y - X\beta)^T (y - X\beta) + \lambda \beta^T \beta$$

- y : $n \times 1$ vector of targets
- X : $n \times p$ feature matrix
- β : $p \times 1$ vector of coefficients
- λ : regularization parameter

Why Ridge Handles Multicollinearity ?

- In OLS, $X^T X$ is singular or near-singular when features are highly correlated.
- Ridge adds λI to diagonal, making $(X^T X + \lambda I)$ strictly positive definite and always invertible.

2. Ridge Normal Equation

- Setting gradient $\nabla_{\beta} L = 0$ gives :

$$\beta_{\text{Ridge}} = (X^T X + \lambda I)^{-1} X^T y$$

Key Properties of Ridge

- Coefficients shrink towards zero (never exact zero).
- Handles multicollinearity (guaranteed invertibility).
- Asymmetric shrinkage : larger weights penalized more (λw_j^2 grows quickly).
- All original features remain in the model.

8. L1 Regularization : Lasso Regression

1. Lasso Loss Function

- Lasso (Least Absolute Shrinkage and Selection Operator) uses an L1 penalty:

$$L_{\text{Lasso}}(w) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |w_j|$$

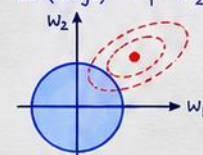
2. Automatic Feature Selection (Sparsity)

- Drives uninformative or redundant feature weights to exact zero ($w_j = 0$).
- Produces a sparse model containing only a subset of features.
- Performs automatic feature selection.

3. L1 vs L2 Geometric Constraint Boundaries

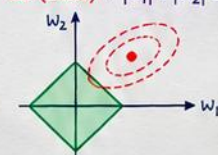
$$L2 \text{ (Ridge)} : w_1^2 + w_2^2 \leq r^2$$

$$L1 \text{ (Lasso)} : |w_1| + |w_2| \leq r$$



Circle / Sphere (smooth boundary)

Typically touches at non-zero point (does not set coefficients to 0).



Diamond / Rhombus (sharp corners)

Intersects at corners $\Rightarrow$ drives some coefficients to exactly 0.

10. Feature Scaling Requirement

- Must standardize features (z-score scaling) before running Ridge or Lasso.
- Without scaling, features with smaller raw scales (e.g. Income in Millions) require larger weights and will be penalized unfairly.

11. Hyperparameter Tuning via Cross-Validation

- Use K-Fold Cross-Validation to select optimal λ^* that minimizes validation MSE.
- Too small λ : high variance, overfitting.
- Optimal λ^* : lowest cross-validation error.
- Too large λ : high bias, underfitting.